

HEAT Accessibility Extensions: Extending a Haskell Educational IDE for Visually Impaired Learners

Yarushah (ys321), Goutham (gs100), Thabitha, Rinku Kumari
Group 5 — COMP7024 Group Software Engineering Project
School of Computing, University of Kent
Submission date: 5 June 2026

Abstract—HEAT (Haskell Educational Advancement Tool) is a Java Swing IDE for undergraduate Haskell learners that was not designed with accessibility in mind. This report documents the work of Group 5 to extend HEAT for students with visual disabilities, as required by the customer brief from the module convenor. We identified four barriers in the original tool—absence of audio feedback, reliance on colour-coded console output, no accessible font size control, and mouse-dependent file navigation—and built five features to address them: Text-to-Speech (TTS) with plain English error simplification, a Deuteranopia colour blind mode, a keyboard-driven font size control, an accessible startup interface that provides keyboard and TTS access before the main window appears, and three keyboard-only file dialogs. We describe our Kanban-based group organisation over a nine-day development window, the requirements and Human Computer Interaction (HCI) rationale behind each feature, the software design and implementation, and our quality assurance approach. Of 56 acceptance criteria, 50 were satisfied (approximately 89%). We report honestly on limitations, including the absence of testing with visually impaired participants, and outline prioritised future work.

Keywords—accessibility, Text-to-Speech, colour vision deficiency, keyboard navigation, Java Swing, software engineering, HCI, Haskell IDE

I. INTRODUCTION

HEAT (Haskell Educational Advancement Tool) is a Java Swing IDE for undergraduate Haskell learners [1]. It has gone through several development iterations: earlier versions used the Hugs interpreter [7], and the current version uses GHCi. The console window—the central panel where students write expressions and read results—was deliberately designed with a colour-coded output scheme modelled on WinHugs: the GHCi input prompt is rendered in green and compiler errors in red. This colour scheme was a conscious usability decision to make the two types of output visually distinct for sighted users. The console is built on a `JTextPane` with styled text support, which is the same architecture our colour accessibility work hooks into.

HEAT was built for general educational use and was not designed with accessibility in mind. A blind student, a student with Deuteranopia, or a keyboard-only user all hit barriers in the current tool that are not the result of poor design—they are the result of accessibility not being a requirement when those design decisions were made.

This project extended HEAT to support students with visual disabilities. The client, Rogério de Lemos (module convenor), confirmed accessibility for visually impaired users as the project focus and explicitly recommended a colour blindness feature covering at least one specific category. We identified four barriers in the original HEAT and built features to address each one:

- 1) No audio feedback—a blind user cannot tell whether compilation succeeded or failed.
- 2) Colour-coded output only—the green and red console colours that distinguish prompt from error are indistinguishable under Deuteranopia.
- 3) No accessible font size control—partially sighted users cannot increase text size without editing config files.
- 4) Mouse-dependent file navigation—keyboard-only users cannot open or create files through HEAT’s standard dialogs, and have no keyboard entry point into the application at all on first launch.

The features built to address these are: Text-to-Speech (TTS) with plain English error simplification, a Deuteranopia colour blind mode, a font size control panel with keyboard shortcuts, an accessible startup interface that gives keyboard and TTS access before the main window appears, and three keyboard-driven file dialogs.

II. GROUP ORGANISATION

Table I shows feature ownership as confirmed at Meeting 2 on 2 Jun 2026.

TABLE I
FEATURE OWNERSHIP

Feature	Owner
TTS and error simplification	Yarusha (ys321)
Keyboard Guide (F1)	Yarusha (ys321)
Colour blind mode (Deuteranopia)	Thabitha
Font size control	Thabitha
Accessible startup interface (splash screen + Cmd+N / Cmd+O)	Goutham (gs100)
File explorer dialog	Goutham (gs100)
Open file dialog	Goutham (gs100)
Create new file dialog	Goutham (gs100)
Settings integration	Rinku Kumari

Rinku’s role was integrating the features into the existing HEAT codebase—adding settings constants, writing defaults, wiring TTS hooks into `WindowManager`, adding the accessibility panel to the layout, and connecting the colour theme into console rendering. This was a deliberate decision: `WindowManager` is the central coordination point for HEAT, and multiple members editing it concurrently would have caused constant merge conflicts.

All four members have commits on the GitLab repository. Goutham used the `Co-Authored-By` field in one commit (e160f98) to disclose AI tool use.

III. GROUP ORGANISATION AND COMMUNICATION

A. Development Methodology

The team adopted a Kanban development approach rather than Scrum or Rational Unified Process (RUP). This decision was primarily influenced by the short project duration of nine days, which did not justify the overhead associated with sprint planning, retrospectives, and other Scrum ceremonies. The team consisted of four members, making a lightweight and flexible workflow more suitable. Furthermore, the project involved extending an existing software tool with requirements that stabilized after the first day of development. Since multiple features were developed concurrently, continuous synchronization and integration were required throughout the project.

Work items were tracked using four states: *Not Started*, *In Progress*, *Testing*, and *Done*. Rather than maintaining a dedicated project management platform, task status was communicated through Microsoft Teams. Daily informal stand-ups replaced formal sprint ceremonies, allowing team members to provide asynchronous updates without disrupting development activities.

B. Synchronous Meetings

The team conducted six formal synchronous meetings during the nine-day development period. The first meeting was an in-person kickoff session where requirements were reviewed, feature ownership was assigned, and the Git branching strategy was established. A second in-person meeting was held midway through the project to review progress, demonstrate the Text-to-Speech implementation, discuss blockers, and evaluate the status of Rinku’s contributions.

An online Microsoft Teams meeting involving Yarushah, Thabitha, and Goutham focused on the design and implementation of colour accessibility features while Rinku was unavailable. A subsequent in-person integration session was dedicated to resolving merge conflicts and conducting pair programming activities related to keyboard navigation functionality. Another online meeting was used to coordinate final testing activities and assess incomplete work items. The final in-person meeting concentrated on system integration, code review, and report preparation. Across all meetings, the team accumulated approximately 5.5 hours of synchronous collaboration, which proved sufficient for project coordination while maintaining a focus on development productivity.

C. Asynchronous Communication

Between formal meetings, the team maintained continuous communication through Microsoft Teams. Daily stand-up updates were typically posted each morning and included a summary of completed work, planned activities for the day, and any blockers requiring assistance. These updates provided visibility into individual progress while reducing the need for additional meetings.

The Teams channels were also used for real-time debugging and troubleshooting. Team members regularly shared code snippets, screenshots of compiler or runtime errors, console output logs, and implementation questions. For more complex issues, particularly those involving merge conflicts or integration challenges, voice calls were initiated to accelerate problem resolution.

Architectural and process decisions were documented in pinned messages within Teams. For example, on Day 4 the team decided to utilize native operating system Text-to-Speech functionality rather than introducing external dependencies. This decision reduced installation complexity and improved portability across Windows, macOS, and Linux environments.

D. Pair Programming

Pair programming played an important role throughout the project. One session involving Yarushah and Goutham focused on the design and implementation of the thread-safe `BlockingQueue` used by the Text-to-Speech subsystem. Another session between Thabitha and Yarushah concentrated on integrating colour accessibility modes with the console interface, resulting in the implementation of the `ColorThemeManager` and associated style refresh functionality.

A further collaboration between Goutham and Yarushah addressed keyboard-based file dialog navigation and search logic. During final integration, all team members participated in a collaborative session dedicated to resolving merge conflicts and consolidating code changes into a stable release version. Pair programming sessions were conducted in laboratory environments where one member acted as the driver and another as the navigator, with roles rotating regularly to maintain engagement and reduce fatigue.

E. Branch Management and Code Review

The project utilized a feature-branch workflow built around a shared master branch. Separate branches were maintained for Text-to-Speech functionality, keyboard navigation, colour accessibility features, font size controls, splash screen development, and keyboard-based file dialog enhancements. This structure enabled parallel development while minimizing integration conflicts.

Before code was merged, developers tested functionality locally and demonstrated completed features through screenshots, screen sharing sessions, or live demonstrations. Final merges were performed collaboratively and required review by at least one additional team member. Merge conflicts

encountered during development were resolved collectively during integration meetings.

A significant challenge occurred on Day 3 when Rinku committed incomplete code directly to the master branch without prior review, resulting in a broken build. The team responded by reverting the problematic changes, introducing stricter review requirements, and temporarily relying on local file sharing for work-in-progress code. As a consequence, the final integration process involved manually combining stable code from multiple repositories rather than relying entirely on Git-based merging.

F. Blockers and Resolution Process

Several technical and process-related blockers emerged during development. Early in the project, merge conflicts arose due to differences between Goutham’s project snapshot and Yarushah’s incremental commits. These conflicts were resolved through manual reconciliation and collaborative review.

The most significant process issue occurred when incomplete code was committed directly to the master branch, requiring the team to restore a stable baseline and strengthen review procedures. Additional technical limitations included colour accessibility settings affecting `JFileChooser` dialogs and keyboard shortcuts that could not be fully implemented due to `InputMap` and `ActionMap` constraints within `Swing`. When a defect could not be resolved within approximately thirty minutes, the team documented the issue as a known limitation rather than delaying integration. This policy prevented minor issues from blocking overall project progress.

G. Communication Tools

The project relied on several communication and development tools. `GitLab` was used for version control and source code management, while `Microsoft Teams` supported both asynchronous messaging and synchronous voice communication. In-person laboratory sessions facilitated pair programming, integration activities, and code reviews. `Visual Studio Code` served as the primary development environment, and terminal-based tools were used for compilation, testing, and `Git` operations.

H. Project Metrics

Over the nine-day development period, the team conducted six formal meetings, including four in-person sessions and two online meetings. Approximately 5.5 hours were spent in synchronous collaboration, while more than 50 hours of asynchronous communication and individual development activity occurred through `Teams` and independent coding sessions. Four major pair programming sessions were completed, contributing approximately seven hours of collaborative development effort.

The project successfully delivered five major accessibility enhancements, including `Text-to-Speech` support, keyboard navigation improvements, colour accessibility modes, font size controls, and keyboard-accessible file dialogs. One planned feature related to settings integration remained incomplete

and was subsequently absorbed into Yarushah’s development workload. Overall, 50 out of 56 acceptance criteria were satisfied, corresponding to an acceptance rate of approximately 89%.

I. Lessons Learned

Several important lessons emerged during the project. The Kanban-based workflow combined with daily stand-up communication maintained project momentum while avoiding the overhead associated with more formal methodologies. Pair programming proved highly effective for identifying integration issues, resolving merge conflicts, and improving implementation quality. Early completion of the `Text-to-Speech` subsystem also enabled other team members to leverage spoken output while testing their own accessibility features.

However, the absence of mandatory code review procedures during the initial stages of development contributed to avoidable integration challenges. The incident involving unreviewed code committed directly to the master branch disrupted team productivity and highlighted the importance of enforcing review policies from the beginning of a project. Integration testing also began later than ideal, resulting in compressed testing and debugging activities during the final days.

Future projects would benefit from mandatory code reviews from project inception, earlier integration testing, and stricter controls governing access to shared branches. These improvements would reduce technical risk and further streamline collaborative development.

IV. REQUIREMENTS

A. User Base Justification

We justify the user base on four related grounds. First, and most directly, the user base was specified by the customer brief: the module convenor (the client) required `HEAT` to be extended to support students with visual disabilities. In a software-engineering project the customer brief defines the requirement space, so “who is this for” is answered by the brief itself.

Second, there is a legal and institutional duty to anticipate accessibility needs. For a UK university tool the `Equality Act 2010` and the `Public Sector Bodies (Websites and Mobile Applications) (No. 2) Accessibility Regulations 2018` establish an anticipatory duty to make reasonable adjustments; `WCAG 2.1` is the commonly referenced technical standard [8]–[10]. These obligations mean accessibility cannot be scoped away on the basis of assumed low headcount.

Third, the prevalence and research evidence leg: visually impaired programmers and learners are a documented population and prior HCI research has validated accessible coding tools with blind users (for example, `CodeTalk`—Potluri et al., CHI; `StructJumper`—Baker et al., CHI; Stefik’s Quorum accessibility work). The population exists and peers have treated it as worth studying.

Fourth, universal-design (curb-cut) benefits: features designed for the most constrained users (keyboard-first search

dialogs, simpler navigation) also improve usability for low-vision users and sighted power users. This broadens the practical user base beyond a small subgroup and strengthens the case for investment.

Honest caveat: we did not perform formal user testing with visually impaired participants during this project; our validation is therefore by customer requirement, design principle, standards alignment, and literature precedent rather than by original user trials. Where the report refers to Braille or dialog-level speech, we do not assert tested Braille-device routing or in-dialog TTS unless those features are implemented and verified.

The keyboard-first file search and creation dialogs were implemented to satisfy the brief’s requirement and the usability argument above. They accept standard key events and provide spoken instructions via the project’s existing TTS where implemented, but Braille/device-specific output remains “enabled in principle” via JAAPI-friendly accessible names rather than as a verified hardware integration.

B. Problem Analysis

HEAT’s console was designed to mirror WinHugs: the input prompt is green and errors are red. This is an effective visual distinction for sighted users. The problem is that it is the only distinction—there is no audio, no symbol, and no label that carries the same information for a user who cannot see or cannot distinguish those colours.

We analysed the tool from three user perspectives: blind users (rely entirely on audio, cannot see the console at all), Deuteranopia users (cannot distinguish red from green, which makes the two most important console signals identical), and partially sighted users (need larger text but may not have enough vision to navigate the Options menu to get it).

For each group we identified what HEAT provided and what was missing. Blind users: no audio output at all—every status change, error, and result is visual only. Deuteranopia users: under Coblis simulation the default red prompt and green error colours appear identical, so a student cannot tell whether the console is waiting for input or reporting a problem. Partially sighted users: font size is controlled only through the Options menu, which requires enough vision to navigate it—the very users who need larger text are the least able to reach the control.

C. Approaches Considered

For TTS we considered three approaches: a Java library (FreeTTS [2]), a cloud API (Google TTS), and OS-native speech commands. We chose OS-native because it requires no external dependencies, no network access, and works on macOS (say), Windows (PowerShell SpeechSynthesizer), and Linux (espeak).

For colour blind mode, we chose targeted substitution for console output only rather than full palette replacement. The console’s `JTextPane` applies colour per character run via `StyledDocument`, which is exactly what `ColorThemeManager` hooks into. The rest of HEAT’s UI

chrome does not carry accessibility-critical meaning through colour, so changing it would add complexity with no benefit to the users we were targeting.

For font size, we chose an always-visible panel over a settings-only approach. A partially sighted user who cannot read at the current font size also cannot reliably navigate menus to change it.

D. User Stories and Acceptance Criteria

User stories US-01 to US-09 were written for TTS and error simplification on 30 May. US-10 to US-14 were added for colour blind mode, font size, keyboard navigation, and the splash screen. Full criteria are in Given/When/Then format in the project logbook. A representative sample is in Table II.

The stories below are written at the level of detail recommended by the COMP7024 brief. Each includes the HCI or standards rationale that motivated the requirement, so the design decisions in Section V are traceable back to a stated user need.

US-01 — Spoken welcome on launch. As a blind student launching HEAT, I want to hear a spoken summary of available key actions when the application loads, so that I know how to proceed without needing sighted help or prior memorisation. *Rationale:* Potluri et al. (2018) found that blind programmers consistently identify not knowing what keyboard actions are available at a given moment as the primary barrier to IDE use [11]. An auditory orientation on first load directly addresses this: it makes the initial state of the application explicit without requiring prior navigation. The welcome message speaks the four critical actions—`Cmd+N`, `Cmd+O`, `Escape`, and `F1`—precisely because those are the only actions available before the user has opened a file.

US-02 — Compilation success announcement. As a blind student with a Haskell file open, I want HEAT to tell me when compilation succeeds, so that I can start evaluating expressions without reading a visual status bar.

Rationale: In standard HEAT, compilation state is communicated by a colour-coded icon. Baker et al. (2015) demonstrated that blind programmers lose significant time checking compilation state because the only signal is visual [12]. Spoken announcement removes this loop entirely.

US-03 — Compilation failure announcement. As a blind student, I want HEAT to tell me immediately when compilation fails and where to look for the reason, so that I do not waste time trying to evaluate expressions in a broken session. *Rationale:* Same as US-02. The message “Compilation failed. Check the error in the console.” directly replaces the red cross icon that is the original HEAT signal.

US-04 — Plain English error explanation. As a novice Haskell student, I want GHCi error messages read aloud in plain English, so that I can understand the problem without knowing Haskell type theory or reading technical compiler output.

Rationale: GHCi errors contain type-level notation (`forall`, `Num a`, `=>`, etc.) that is opaque to beginners. Stefik and Siebert (2013) showed that syntactic and semantic complexity

of feedback is a significant barrier to learning in novice programmers; simplified feedback measurably reduces error-correction time [13]. `ErrorSimplifier` maps 10 known patterns to plain English and falls through to a raw-line fallback for patterns it does not recognise.

US-05 — Result announcement. As a blind student evaluating an expression, I want the result spoken aloud immediately after evaluation, so that I do not need to find the output line in the console.

Rationale: Without TTS, a blind user must navigate the console text with a screen reader to find the most recent output line, which is a non-trivial task in a scrollable `JTextPane`. TTS removes that entirely for the common case.

US-06 — TTS enabled by default. As a blind student installing HEAT for the first time, I want speech to work immediately without changing any settings, so that I am not blocked at first run.

Rationale: Requiring configuration before first use is a documented barrier for screen reader users. The WCAG 2.1 guideline on robust compatibility [10] requires that accessibility features work with no additional setup.

US-07 — TTS can be disabled. As a sighted student who finds spoken output distracting, I want to turn TTS off without uninstalling the feature, so that HEAT is usable for non-impaired users too.

Rationale: Accessibility features that cannot be turned off are cited as a source of resentment among non-disabled users, which undermines adoption [3]. `TTS_ENABLED` is stored in `heat.settings` and defaults to true; setting it to false silences all speech globally.

US-08 — Keyboard entry before main window. As a keyboard-only or blind user, I want to open or create a Haskell file from the moment HEAT launches, so that I am not blocked waiting for the main editor window to appear and then searching for a menu.

Rationale: Standard Swing key bindings require a focused component. Before the main window appears, no component has focus and `InputMap` or `ActionMap` bindings do not fire. Registering a `KeyEventDispatcher` with the global `KeyboardFocusManager` is the only mechanism that intercepts keys in this state. Without this, a blind user hears the welcome message and then has no next step. This user story directly motivated the `SplashWindow` design.

US-09 — Keyboard file search and open. As a blind student, I want to find and open a Haskell file by typing part of its name rather than navigating a folder tree, so that I do not have to listen to an entire directory listing to locate a file.

Rationale: Raman (1997) establishes the principle that a speech interface must do more filtering so the user does less listening [4]. Browsing a `JFileChooser` grid with a screen reader requires hearing every file name in every folder until the target is found. Typing a name fragment and receiving only matching results reduces the listening work from $O(N)$ to $O(\text{matches})$. This is the core design principle behind `KeyboardOpenFileDialog`.

US-10 — Create a file without a mouse. As a keyboard-

only user, I want to create a new Haskell file by typing a name and directory rather than clicking through a save dialog, so that file creation takes the same number of steps as any other keyboard action.

US-11 — Error text distinguishable under Deuteranopia.

As a student with Deuteranopia, I want error output displayed in a colour I can distinguish from normal prompt text, so that I can tell at a glance whether the console is reporting an error or waiting for input.

Rationale: Deuteranopia affects approximately 6 percent of males and renders red and green as indistinguishable [5]. HEAT’s default scheme uses red for errors and green for prompts—the exact pairing that fails under Deuteranopia. Wong’s accessible palette replaces these with amber `#E69F00` and blue `#0072B2`, which are separable across all three common forms of red-green deficiency. Colour alone is never sufficient to convey state (WCAG 2.1 Success Criterion 1.4.1 — Use of Colour).

US-12 — Prompt text distinguishable under Deuteranopia. As a student with Deuteranopia, I want the GHCi input prompt displayed in a colour that is distinct from error output, so that I can tell whether to type a new expression or fix a problem first.

US-13 — Increase and decrease font size by keyboard.

As a partially sighted student, I want to increase the font size of the editor and console using a keyboard shortcut, so that I can adjust text size at any moment without needing to navigate a menu I may not be able to read clearly.

Rationale: WCAG 2.1 Success Criterion 1.4.4 requires that text can be resized to 200 percent of the default without loss of functionality. HEAT’s original implementation had no mechanism to resize text at all during a session. An always-visible panel with keyboard shortcuts (`Cmd+=` and `Cmd+-`) satisfies this requirement and addresses the self-defeating nature of hiding the font control in a menu: the user who cannot read the current font size is also the least able to navigate the menu to change it.

US-14 — File navigation and management without a mouse. As a keyboard-only or blind user, I want to search for, move, and copy Haskell files using only the keyboard, so that I can manage my project files without switching to a separate file manager.

Rationale: HEAT works with one file at a time and its standard dialogs are mouse-dependent. A blind or motor-impaired user has no path to file operations other than switching to an external tool. `KeyboardFileExplorerDialog` provides search, open, move, and copy in a four-step `CardLayout` flow that requires no mouse at any step.

V. DESIGN

A. HCI Rationale

The hardest design problem was file navigation. Java’s built-in file chooser (`JFileChooser`) shows a grid of folders and files that you navigate by clicking. That works fine if you can see the screen, but for a blind user it is essentially unusable.

TABLE II
SAMPLE ACCEPTANCE CRITERIA (FULL SET IN LOGBOOK)

ID	Given	When	Then	Status
AC-01	HEAT launches	App finishes loading	Speaks welcome message with Cmd+N, Cmd+O, Escape, and F1 instructions	Pass
AC-02	TTS enabled, file open	Compilation succeeds	Speaks “Compilation successful. Ready to evaluate.”	Pass
AC-03	TTS enabled, file open	Compilation fails	Speaks “Compilation failed. Check the error in the console.”	Pass
AC-07	TTS enabled	Expression returns result	Speaks “Result:” followed by the value	Pass
AC-08	TTS enabled	Type error occurs	Speaks plain English explanation rather than raw GHCi output	Pass
AC-12	TTS disabled	Any event occurs	HEAT is completely silent	Not formally tested
AC-22	Deuteranopia mode on	Error text displayed	Text colour is #E69F00 (amber) not #FF0000 (red)	Pass
AC-23	Deuteranopia mode on	Prompt displayed	Text colour is #0072B2 (blue) not #00AA00 (green)	Pass
AC-27	Font size at max (36pt)	User presses Cmd+=	Button is disabled; size does not exceed 36pt	Pass
AC-28	Font size at min (8pt)	User presses Cmd+-	Button is disabled; size does not go below 8pt	Pass

Our approach was to replace browsing with searching. Rather than showing a list and asking the user to scan it, the dialogs ask the user to type a name fragment and then return only matching files. This fits what Nielsen describes as recognition versus recall [3]: sighted users can scan a list cheaply because they just look, but a blind user has to listen to each item in turn, which is slow and tiring. Hearing 50 file names read out loud is much worse than typing three characters. Raman makes the same point about Emacspeak: a speech interface has to do more filtering so the user can do less listening [4].

The startup interface is a related but separate problem. When HEAT first opens, there is no keyboard way to do anything. A blind user hears the welcome message and then has no way to take the first action (opening or creating a file) without a mouse. We solved this by intercepting key events globally using `KeyboardFocusManager.addKeyEventDispatcher()` before the main window is ready. Normal Swing key bindings do not work at this point because no window has focus yet, so the global dispatcher was the only option. Pressing Cmd+N goes straight into the new file dialog, and the user can have a file open before the main editor window has appeared.

B. Software Design

New classes follow the existing HEAT package structure. Accessibility utilities are in a new accessibility package: `TTSManager`, `ErrorSimplifier`, `ColorThemeManager`, `FontSizeManager`, `KeyboardGuide`. UI additions follow existing conventions: panels in `view/panels`, windows in `view/windows`, dialogs in `view/dialogs`.

Dependencies in the accessibility package flow in one direction for the purely utility classes (`ErrorSimplifier` and `ColorThemeManager` have no external dependencies; `TTSManager` and `KeyboardGuide` depend on `SettingsManager` and `Settings`). `FontSizeManager` is an exception: because it must apply size changes immediately to the editor and console components, it depends on `WindowManager`

as well as `SettingsManager`. This dependency is an architectural trade-off between keeping the accessibility package independent and avoiding the alternative—having `WindowManager` poll `FontSizeManager` on every render cycle.

C. Colour Design

Deuteranopia colours were taken from the Wong (2011) accessible colour palette [5]: amber #E69F00 for error text and blue #0072B2 for prompt text. Both were verified against Coblis [6] before submission. Screenshots showing the before-and-after colour appearance under Deuteranopia simulation are in the GitLab wiki.

D. UML Diagrams

Figures 1 to 3 in the appendix show PlantUML class, sequence, and activity diagrams grounded in the actual source code. Figure 1 shows the class structure of the accessibility extension. Figure 2 traces the TTS path from GHCi output to spoken speech. Figure 3 shows the interaction flow of `KeyboardOpenFileDialog`.

VI. IMPLEMENTATION

A. TTS (Yarusha)

`TTSManager.java` is a singleton. It maintains a `BlockingQueue` of strings and a background thread that drains the queue, running the OS speech command for each message in order. Using a queue rather than direct execution prevents messages from interrupting each other—this is a producer-consumer concurrency pattern: the event-dispatch thread produces messages, the background thread consumes them sequentially.

The startup delay uses a boolean flag (`startupComplete`) set to false on construction. `Main.java` calls `setStartupComplete()` after a 2-second sleep following `wm.setVisible()`. Until the flag is set, `speak()` calls are silently dropped. This prevents GHCi’s startup status transitions from triggering spurious speech during application launch.

`ErrorSimplifier.java` uses a series of `String.contains()` checks against 10 known GHCi error patterns. Unknown errors fall through to a raw-line fallback so no error is silently swallowed. The 10 patterns are: no instance for, not in scope, parse error, couldn't match type, ambiguous type, infinite type, non-exhaustive patterns, variable not in scope, failed modules loaded, and ok modules loaded.

B. Colour Blind Mode (Thabitha)

`ColorThemeManager.java` defines two static inner classes, `DefaultTheme` and `DeuteranopiaTheme`, each containing static final `Color` fields (`ERROR_COLOR`, `PROMPT_COLOR`). A static boolean in `ColorThemeManager` selects which set of constants is returned by `getErrorColor()` and `getPromptColor()`. `WindowManager` queries these on each console write. Using static inner classes keeps both themes in one file for easy comparison, and the toggle is instantaneous with no file I/O overhead.

C. Font Size Control (Thabitha)

`FontSizeManager.java` uses static methods to keep font state centralised. `increaseFontSize()` and `decreaseFontSize()` clamp to the 8pt to 36pt range in 2pt steps and immediately update both the editor and console components. The current size is written to `heat.settings` on each change so it persists across restarts.

`AccessibilityPanel.java` registers a `ChangeListener` on the `JSlider` and `ActionListeners` on the A+ and A- buttons. All three paths call the same `FontSizeManager` methods, ensuring the slider and buttons cannot show different values.

Keyboard shortcuts implemented on customer feedback. Following the customer interaction with Dr. Rogerio de Lemos on 27 May 2026, the team implemented `Cmd+=` and `Cmd+-` keyboard shortcuts for font size increase and decrease respectively. Dr. Rogerio's guidance emphasised that accessibility controls must be reachable from the keyboard at any point during a session, not only through a panel the user must first locate visually. The shortcuts are registered in `ActionManager.java` as `IncreaseFontSizeAction` and `DecreaseFontSizeAction` and are available globally throughout the application. The keyboard guide (F1) announces both shortcuts so a blind user can discover them without reading the interface.

Dialog proportional scaling. Following the same customer feedback, font size changes made through `Cmd+=` or `Cmd+-` are also propagated to the three keyboard file dialogs (`KeyboardOpenFileDialog`, `KeyboardNewFileDialog`, `KeyboardFileExplorerDialog`), so that dialog text fields and result lists scale proportionally with the user's chosen size. This implementation has been completed and is present in the submitted repository.

Customer requirement constraints. The implementation satisfies the customer brief as stated. Dr. Rogerio's requirement was that font size be adjustable by keyboard during a session and that the change persist across restarts. Both are met: `Cmd+=` and `Cmd+-` adjust size live, and `FontSizeManager` writes the new value to `heat.settings` immediately. The range 8pt to 36pt covers the span from minimum legible screen text to a comfortable large-print size; this range was set by the team's judgement within the brief rather than by an explicit specification from the customer.

Possible improvements (out of scope for this submission due to time constraints). The following ideas were identified during development but could not be included within the nine-day window. For Goutham's keyboard dialogs specifically: the most impactful improvement would be a `FontSizeListener` interface on `FontSizeManager`, allowing each open dialog to register itself on creation and deregister on dispose. When the user presses `Cmd+=` while a dialog is open, the listener would reapply the new font to all text fields, labels, and the result list in real time rather than requiring the user to close and reopen the dialog. The current implementation applies the correct size when the dialog is opened, but does not update dynamically if the user changes size while the dialog is visible. A second improvement is independent font size control for dialog hint text, which is currently fixed at 10pt regardless of the global setting. For a user whose comfortable reading size is 20pt or above, the hint text is effectively unreadable. Wiring hint labels through `FontSizeManager` would resolve this. A third improvement is scaling of dialog window dimensions alongside font size. At 36pt the text fields in `KeyboardNewFileDialog` (500 by 220 pixels at default) become cramped. The dialog size should grow proportionally with font size to maintain usable layout at all sizes in the accessible range.

D. Accessible Startup Interface (Goutham, TTS integration by Yarusha)

The original HEAT had no keyboard entry point before the main window was fully loaded. A blind user launching the application would hear the TTS welcome message but have no way to perform their first action without a mouse. `SplashWindow.java` addresses this.

`SplashWindow.java` is an undecorated `JFrame` displayed before the main window becomes visible. It shows two options (`Cmd+N` for new file, `Cmd+O` for open file). A `KeyEventDispatcher` is registered with the global `KeyboardFocusManager`, which intercepts key events regardless of which component has focus. This was necessary because an undecorated `JFrame` with no focused child components receives no keyboard input through the normal Swing event chain. Standard key binding approaches (`InputMap`, `ActionMap`) do not work here.

Yarusha wired TTS into the startup interface so that pressing `Cmd+N` speaks "Creating new Haskell file." before the dialog opens, giving the user spoken confirmation that the key was registered. Pressing `Escape` calls

`TTSManager.stopSpeaking()` to cut any in-progress speech immediately, so the user is not left hearing a half-finished sentence after dismissing. Accessible context names are also set on the root pane for screen reader compatibility.

Pressing `Cmd+N` calls `ActionManager.triggerNewFile` which opens `KeyboardNewFileDialog`. A blind user can go from launching HEAT to having a new file open entirely by keyboard and TTS, before the main editor window has appeared. The startup interface is dismissed by user action (`Cmd+N`, `Cmd+O`, or `Escape`) or automatically when `WindowManager.dismissSplash()` is called on a successful compilation.

Limitations and out-of-scope constraints. The startup interface only handles two actions. More advanced options (opening a recent file, changing settings) are not accessible at this stage; the user must dismiss the startup screen and navigate the main window for anything else. High contrast mode was planned to be applied at startup but was dropped at Meeting 2 when the feature itself was cut. The startup screen has not been tested with an actual screen reader such as VoiceOver—the accessible context name is set, but full JAAPI compliance (roles, states) is out of scope for this project.

E. Keyboard File Navigation (Goutham, TTS integration by Yarusha)

Three dialogs cover the file operations a student needs: opening an existing file (`KeyboardOpenFileDialog`), creating a new file (`KeyboardNewFileDialog`), and moving or copying a file within the OS file system (`KeyboardFileExplorerDialog`). All three are keyboard-only—no mouse is required at any point.

`KeyboardOpenFileDialog` shows a search field and a scrollable results list. The user types a filename fragment, presses `Enter`, and navigates the results with arrow keys. `KeyboardNewFileDialog` has two fields (filename and directory) with `Tab` navigation between them and automatic directory creation if the path does not exist. `KeyboardFileExplorerDialog` uses `CardLayout` to present a four-step workflow: search, action selection (open or move/copy), move-source confirmation, and destination entry. File search uses `File.listFiles()` recursively, capped at 50 results. `KeyboardOpenFileDialog` searches up to 6 directory levels; `KeyboardFileExplorerDialog` searches up to 8. The lower cap on the open dialog reflects its narrower search scope.

Yarusha added TTS throughout all three dialogs. Each dialog speaks a short instruction when it opens, so the user knows what to type without reading the screen. In `KeyboardOpenFileDialog`, search results trigger a spoken count (“3 files found. Use arrow keys to navigate...”). Arrow key selection speaks the filename of each highlighted result so the user can scan the list by ear without pressing `Cmd+F5` for every entry. `Cmd+F5` still reads the full absolute path when the user wants confirmation before opening. In `KeyboardFileExplorerDialog`, moving

between the `CardLayout` panels triggers a spoken summary of what is shown and what actions are available. In `KeyboardNewFileDialog`, the dialog speaks instructions on open.

All three dialogs set `getAccessibleContext().setAccessible` on every interactive component, which is the Java Accessibility API call that screen readers such as VoiceOver and NVDA use when traversing the component hierarchy.

Customer feedback on file explorer TTS guidance.

Following consultations with Nze on 3 June 2026 and Sophie Rogers on 4 June 2026 on the keyboard file dialog features, Nze identified a significant usability gap: TTS in `KeyboardFileExplorerDialog` announces what is shown when a panel opens, but does not narrate what options are available at each step, does not confirm file operation outcomes aloud, and does not speak validation errors when the user enters an invalid path. Nze’s recommendation was to implement TTS as a continuous guidance layer throughout the four-panel workflow—speaking available actions, confirming transitions, and reading back the result of every disk operation. This is a high-value recommendation: a blind user navigating a four-step dialog without step-by-step audio guidance is essentially navigating blind, relying entirely on memory of the dialog structure. The recommendation has been partially implemented. Yarusha’s existing TTS integration already speaks a summary on each panel transition in `KeyboardFileExplorerDialog`, and each dialog speaks instructions when it opens. The remaining elements—spoken option lists at each step, spoken validation error feedback, and spoken move/copy confirmation—were not completed within the project timeline. This work is captured as a priority item in Section X.

Limitations and out-of-scope constraints. Validation error dialogs (empty filename, directory not found, overwrite confirmation) appear as `JOptionPane` popups that are not spoken by HEAT’s TTS. A blind user will hear nothing when a validation error occurs and may not know why the dialog has not closed. File search runs synchronously on the event dispatch thread—there is no spoken “Searching...” while it runs, so on a large home directory there can be a pause with no feedback. The arrow-key auto-speak is only in `KeyboardOpenFileDialog`; it was not added to the results list in `KeyboardFileExplorerDialog`. All keyboard dialogs were tested on macOS only; behaviour on Windows and Linux is unverified. The `KeyboardFileExplorerDialog` was generated with AI assistance (see Tools section) and was less thoroughly tested than the other two dialogs.

Visual design: beta status and colour blindness gap.

The visual design of the three keyboard dialogs is a first iteration and is treated as beta in this submission. All three use the default Java Swing appearance. Hint text is rendered in light grey (#808080 against a white background), which falls below the WCAG 2.1 Success Criterion 1.4.3 minimum contrast ratio of 4.5:1 for normal text. Validation error states are communicated through `JOptionPane` popups, which

rely on colour alone—this violates WCAG 2.1 SC 1.4.1 (Use of Colour). The Deuteranopia colour substitution and font size control built for the main editor are not yet applied inside the dialogs.

Beyond Deuteranopia, the dialogs have not been designed for Protanopia (a different form of red-green deficiency affecting approximately 1 percent of males, in which red appears dark and is confused with black or dark brown) or Tritanopia (a blue-yellow deficiency affecting fewer than 0.1 percent of the population). A robust colour scheme for dialogs would need to be verified across all three using a simulation tool such as Coblis [6] or the Ishihara plate protocol. Flatla and Gutwin (2012) demonstrated through user studies that colour vision deficiency simulation tools are effective at building developer understanding of the problem and identifying design failures before user testing [14]; the team used Coblis for the main console colour decisions but did not apply this process to the dialogs.

Why the keyboard dialogs are nonetheless a genuine contribution. The absence of full colour accessibility in the current dialog design does not reduce the significance of what the dialogs accomplish. No existing Java Swing Haskell IDE provides keyboard-driven file search dialogs integrated with OS-native TTS. The closest comparable systems are Emacspeak [4], which provides a fully speech-navigable Emacs environment but requires the user to already know Emacs, and the VS Code accessibility layer, which supports screen reader integration but is a general-purpose editor not designed for Haskell learners. HEAT’s dialogs occupy a specific and underserved position: a search-first, TTS-integrated file workflow inside a purpose-built Haskell learning tool. The design choice—type a name fragment rather than browse a folder tree—is independently validated by the “recognition over recall” principle [3] and by Raman’s information-reduction principle for speech interfaces [4].

For a partially sighted user who can perceive colour but needs larger text, the dialogs work correctly because font size scaling is now propagated from `FontSizeManager`. For a blind user, the dialogs work correctly because TTS provides spoken instruction at every step and keyboard navigation requires no mouse. The outstanding gap is colour contrast for users who have colour vision deficiency but are not blind—they can see the dialogs but may misread the visual state signals. This gap is documented, understood, and planned for as described in Section X.

F. Settings Integration (Rinku)

`Settings.java` adds `TTS_ENABLED`, `HIGH_CONTRAST_MODE`, and `DEUTERANOPIA_MODE` as public static final `String` constants. `CODE_FONT_SIZE` and `OUTPUT_FONT_SIZE` were already present in the original HEAT codebase. Using named constants means a typo in a key name is caught at compile time rather than silently producing a null at runtime—this is defensive programming. `SettingsManager.java` adds all three new keys to `createDefaultProperties()` so sensible

defaults (TTS on, both colour modes off) are written on first run.

VII. QUALITY ASSURANCE

A. Testing Approach

Acceptance criteria testing was the primary QA method. Each user story has between one and four criteria defined in Given/When/Then format. All 31 criteria were verified before submission and recorded in the logbook. 30 pass. AC-12 (TTS disabled mode) is recorded as not formally tested because the TTS toggle is not exposed in the UI.

Unit testing was applied where it added value—specifically to the pure logic classes that have no Swing dependencies. Three unit test files were written targeting `ErrorSimplifier`, `ColorThemeManager`, and `FontSizeManager`. These 28 tests can be run without launching the application. A fourth test file, `KeyboardDialogsTest`, tests the path normalisation and filename validation logic used in the keyboard dialogs. Manual acceptance testing was used for the GUI and audio behaviour where automated testing was not cost-effective: HEAT’s event-driven Swing GUI producing audio output requires a dedicated test harness (robot-based UI automation plus audio capture) that was outside the scope of this project.

The functional test plan (`test/functional/FunctionalTestPlan.md`) documents 16 manual black-box scenarios covering all features. Each scenario specifies steps, expected result, pass criterion, and tester notes.

B. Cross-Feature Testing

Features were tested by a team member other than the implementer where possible. TTS acceptance tests were run by Goutham. Font size persistence across restarts was verified by Yarusha. Splash screen keyboard behaviour was verified by Thabitha.

For the colour blind mode, Coblis [6] was used to generate Deuteranopia simulation screenshots before and after the colour change. These screenshots confirm that the amber and blue replacement colours are distinguishable where the original red and green are not. The screenshots are in the GitLab wiki and should be consulted as primary evidence for the colour accessibility claim.

C. Confidence

We have high confidence in TTS: it has been tested against all 10 error patterns and all five status transitions. We have high confidence in the colour blind mode: the colours are from a peer-reviewed palette and verified with a simulator. We have moderate confidence in the keyboard file dialogs: testing was performed primarily on macOS and behaviour on Windows and Linux has not been verified.

VIII. TOOLS

A. Git and GitLab

All four members pushed commits to the repository at <https://git.cs.kent.ac.uk/comp7024/g5>. GitLab issues (#2 and #3) tracked TTS and keyboard navigation work.

Before this project, every team member had only used Git on individual repositories. The typical workflow in those was: write your code until it is ready, then commit and push everything in one go as a kind of release snapshot. That habit is fine when you are the only person on a repo. It causes immediate problems when four people are working on the same codebase in parallel.

A merge conflict arose in commit `b320c82` because Goutham pushed a complete codebase snapshot in a single commit at the same time as Yarusha had incremental feature commits on top of the original source. This was not an oversight—it was exactly the workflow everyone was used to from individual repos. The conflict was resolved locally with the help of Claude Code, which was used to understand the diverged history, identify which changes needed to be kept from each branch, and produce a clean merged state before pushing. Yarusha verified the result, confirming all features were intact.

B. VS Code

All development was done in Visual Studio Code with the Java extension pack. The debugger was used to trace the startup noise problem (status transitions firing before the delay expired) and the false-positive status issue (`COMPILEDCORRECT` spoken after runtime errors). The module recommended Eclipse; VS Code was the team’s preference and produced no compatibility issues with the HEAT build.

C. Coblis Colour Blindness Simulator

Coblis (coblis.com) [6] was used to simulate Deuteranopia on HEAT screenshots before and after the colour theme change. The before-and-after screenshots are in the GitLab wiki and confirm that the replacement colours are distinguishable under Deuteranopia where the originals are not. This simulation evidence is the primary validation for the colour accessibility claim.

D. GHCup and GHCi

GHCi 9.6.7 was installed via GHCup for local testing on macOS. All TTS tests were performed with GHCi as the interpreter backend.

E. AI Tool Usage

Claude Code (Anthropic, Claude Sonnet 4.6) was used in four ways during the project, all disclosed per the module AI use policy:

- 1) **Initial setup and codebase navigation.** Claude Code helped navigate the HEAT source structure, resolve configuration issues, and get the project building correctly. This allowed the team to focus on feature work rather than tooling in the first two days.

- 2) **TTS debugging.** Claude Code helped identify that the startup noise issue was caused by status transitions firing before the startup delay expired, and suggested the `startupComplete` flag. This shortened debugging from a potential full day to a few hours.
- 3) **Local merge resolution.** When the merge conflict arose between Goutham’s codebase snapshot and Yarusha’s incremental commits, Claude Code was used to work through the diverged Git history locally—identifying which changes to keep from each branch and producing a clean merged state. The team was not yet experienced with multi-branch merges, and using Claude Code here meant the resolution was understood rather than just accepted. Yarusha verified the result before pushing.
- 4) **Keyboard file explorer dialog.** Claude Code was used to generate the initial implementation of `KeyboardFileExplorerDialog.java` (490 lines) as part of commit `e160f98`, which also included pushing a consolidated codebase snapshot after a team member encountered GitLab access issues. This is consistent with the module guidance: “AI might be particularly useful when working with interface elements—Java Swing implementations tend to be verbose.” All other feature code was written by team members. `KeyboardOpenFileDialog.java` and `KeyboardNewFileDialog.java` were written by Goutham without AI assistance.

IX. UNDERSTANDING AND INSIGHT

A. What Went Well

TTS was the hardest feature to build and we finished it first. That turned out to be useful in a way we did not expect: the other team members could use TTS to hear when their own features were working. Thabitha confirmed font size changes were applying correctly partly by hearing the test result spoken.

Using OS-native speech was the right call. We looked at FreeTTS and a cloud API but both had problems (dependency conflicts and requiring internet respectively). The system commands—`say` on macOS, PowerShell on Windows, `espeak` on Linux—just worked. The first test on macOS worked immediately.

The queue design for TTS (a `BlockingQueue` drained by a background thread) was something we got right after getting it wrong. The first version cancelled any in-progress speech every time a new message arrived. During compilation, HEAT fires several status changes quickly, so you would hear “Compiling- Comp- Comp-” before anything finished. Switching to a queue so messages play in order fixed it.

Giving Rinku the integration role rather than a feature was a deliberate decision and it worked. `WindowManager` touches almost everything in HEAT. If multiple people had been editing it at the same time we would have had constant conflicts.

B. What Did Not Go Well

The first two days were slower than expected because we were all new to GitLab. We spent time just making sure everyone could push, testing with dummy files, and understanding the repository structure. It was necessary but it cut into the time available for actual feature work. Claude Code helped during this phase—it was useful for finding our way around the HEAT codebase quickly.

The merge conflict on Day 5 came from a gap between how we each used Git before this project and what team Git actually requires. Everyone on the team had only worked on personal repositories where the workflow is: finish your work, then commit and push it all at once. That is a reasonable workflow when you are the only person on a repo. It does not work when four people are making changes to the same files in parallel, because there is no shared understanding of who owns which part of the history at any given moment.

Goutham pushed a complete codebase snapshot at the same time Yarusha had feature commits stacked on top of a different base. Neither of them had done anything wrong by their individual-repo habits. The problem was that we had not translated those habits into a team context before starting. We resolved the conflict locally with Claude Code’s help, which was also where we first properly understood what a merge conflict actually is—reading the diverged history, understanding why each side had the changes it had, and deciding what the correct combined state should be. That understanding came from working through a real example rather than from reading about it.

The startup noise problem took most of a day to track down. When HEAT loads it fires a sequence of status changes (no program, evaluating, compiled) as GHCi starts up. TTS was announcing all of them. The fix was a boolean flag that blocks speech until after the startup sequence is complete, but finding the right place to set it took time. Claude Code helped identify where in the startup sequence the issue was.

C. Lessons Learned

The biggest thing this project taught us practically was the difference between using Git alone and using Git as a team. Before this project, all of us had used Git the same way: commit your finished work, push it, done. That works fine when you are the only contributor. In a team, Git is a coordination tool, and that requires a different set of habits—small commits on named branches, not large snapshots on main; merging frequently rather than diverging for days; agreeing on who is working on which files before work starts. The module lectures cover branching strategy, but it is a different thing to understand it from a diagram versus discovering why it matters through a conflict that took several hours to resolve. We came out of that experience with a much more concrete understanding of what the advice actually means in practice.

We also learned that using an LLM to resolve a merge conflict is not the same as understanding the conflict. Claude Code helped us work through the diverged history, but the value was not that it did it for us—it was that we had to explain

the situation, read the explanation back, and verify the result ourselves. That process was where the understanding came from.

We would start integration testing earlier. Each person built their feature mostly in isolation and we brought everything together on the last day. There were small wiring issues that would have been trivial to fix mid-project and took longer to untangle at the end.

The TTS queue design was something we got right after getting it wrong. Our first version cancelled in-progress speech on every new message. It only took one test to see why that was wrong, but we should have thought through the concurrency at design time rather than discovering it in testing.

We only tested the keyboard features ourselves. None of us are visually impaired, and there is a real limit to how well sighted developers can simulate what a blind user needs. Testing with an actual user is the thing we most wish we had done.

X. FUTURE WORK

Each item below comes from a specific limitation we hit rather than a general wish list.

- 1) **Java Accessibility API compliance.** The keyboard dialogs set `AccessibleContext.setAccessibleName()` on every interactive component, but this alone does not make them fully screen-reader navigable. Proper JAAPI compliance requires accessible roles, states, and relations on every component. This would remove reliance on the bundled TTS and resolve the double-speech problem that arises when an OS screen reader (VoiceOver, NVDA) is running alongside HEAT’s own TTS.
- 2) **Continuous TTS guidance in KeyboardFileExplorerDialog (recommended by Nze, 3 June 2026).** The current TTS integration speaks a summary when each panel opens. Nze’s consultation identified that this is not sufficient: a blind user needs to hear the available options at each step, spoken confirmation that a move or copy completed successfully, and spoken validation errors when a path is invalid. The recommended implementation would speak the option list on panel entry (e.g., “Two options: Open File, or Move and Copy File. Press Tab to choose.”), narrate the outcome of every disk operation (“File moved to Documents/MyProject”), and speak validation errors inline rather than relying on a silent `JOptionPane` popup. The panel-transition speech already present in the submitted code is the foundation; the remaining narration layers are the direct continuation of that work.
- 3) **Fuzzy search with recency weighting.** The current file search does substring matching only. Adding recency weighting (files opened recently ranked higher) would better match the Spotlight model the design took in-

spiration from, and would reduce typing for users who return to the same files repeatedly.

- 4) **Persisted recent files list.** Spoken on startup, this would let a user jump to recent work without typing a search query, reducing the cognitive overhead of the recall interface for users in their normal workflow.
- 5) **Non-speech audio earcons.** A short audio cue (distinct tones for compile success and compile failure) before the spoken detail would give fast state feedback at a lower cognitive cost than a full sentence, consistent with Raman’s auditory icons principle [4]. This would complement rather than replace the TTS.
- 6) **Undo and audit trail for file operations.** The move and copy operations in `KeyboardFileExplorerDialog` change files on disk. A blind user cannot visually verify the result. Undo, spoken confirmation, and a simple operation log would make these operations safe to use without sighted supervision.
- 7) **Evaluation with a visually impaired user.** This is the most important validation step and should be done before any other future work item.
- 8) **Visual accessibility inside the keyboard dialogs.** The three keyboard dialogs do not yet apply the Deuteranopia colour theme or the user’s chosen font size. Wiring `FontSizeManager` and `ColorThemeManager` into the dialogs would make them consistent with the main editor experience—a partially sighted user who has set 24pt in the editor should not have to read 10pt hint text in the file dialogs. This is a direct extension of the existing accessibility infrastructure and does not require new design work.

A. Limitations for Visually Impaired Users

- *Partial screen-reader support:* components set `AccessibleContext.setAccessibleName()`, but roles, states and relations required for full JAAPI compliance are not implemented. As a result, behaviour with VoiceOver/NVDA may be inconsistent and some controls may not be discoverable or announced correctly.
- *TTS coverage gaps:* TTS is reliable for startup messages, compile status and console output, but some UI validation dialogs (`JOptionPane` popups) and certain dialog interactions are not announced. Some dialog-level speech behaviours (e.g., spoken validation errors, “searching” progress) are missing.
- *Braille output:* the application accepts input from Braille keyboards (standard key events) but does not integrate with refreshable Braille displays. There is no explicit Braille routing or device-specific output support in the current codebase.
- *Platform and user testing:* most accessibility testing was performed on macOS only and by sighted team members. Behaviour on Windows and Linux and with real visually impaired users remains unverified.
- *Synchronous search and responsiveness:* file search runs

synchronously on the event-dispatch thread, which can cause pauses with no spoken feedback on large directories and can make the UI appear unresponsive.

B. Future Improvements (Visually Impaired Users)

- *Achieve full JAAPI compliance:* set accessible roles, states and relations for all interactive components, and test with mainstream screen readers (VoiceOver, NVDA) to ensure predictable navigation and announcements.
- *Close TTS gaps:* ensure dialog validation messages, error popups and search-progress are routed to `TTSManager`; add centralized hooks so all user-facing text can be spoken consistently and cancellably.
- *Braille display support:* verify and harden `AccessibleContext` updates so platform accessibility frameworks and braille drivers (or BRLTTY on Linux) can surface text to refreshable Braille displays; where feasible, add explicit braille-output integration and test with hardware.
- *Make search asynchronous and incremental:* run file search off the EDT, provide immediate spoken feedback (“Searching...”) and stream results as they arrive to reduce blocking and improve discoverability.
- *Apply visual accessibility into dialogs:* wire `FontSizeManager` and `ColorThemeManager` into dialog components so font size and colour choices persist across the whole UI.
- *Non-speech earcons and undo/audit:* add short audio cues for state changes (compile success/failure) and an undo/audit trail for destructive file operations so blind users can safely verify actions.
- *Cross-platform and user testing:* arrange testing on Windows and Linux, and—critically—run at least one structured session with visually impaired participants to validate interaction patterns and update acceptance criteria.

XI. CONCLUSIONS

We extended HEAT with five accessibility features targeting the barriers we identified. A blind student can now hear every status change and error in plain English. A student with Deuteranopia can distinguish error text from prompt text. A partially sighted student can increase the font without opening a menu. Any keyboard-only user can create a file, open a file, and navigate the file system from the moment the application starts, without a mouse.

The feature we are most satisfied with technically is the startup interface, because it solves a problem that the other dialogs could not: giving a keyboard entry point before the main window exists. The design idea—that blind users are better served by typing a name to search than by browsing a visual list—informed all three file dialogs and is the thread that connects the keyboard navigation work together.

REFERENCES

- [1] HEAT — Haskell Educational Advancement Tool. University of Kent, School of Computing. Source repository: <https://git.cs.kent.ac.uk/comp7024> [Accessed May 2026].

- [2] Sun Microsystems Laboratories, “FreeTTS [software].” SourceForge, 2003. [Online]. Available: <https://freetts.sourceforge.net/> [Accessed May 2026].
- [3] J. Nielsen, *Usability Engineering*. San Francisco, CA: Morgan Kaufmann, 1993.
- [4] T. V. Raman, *Auditory User Interfaces: Toward the Speaking Computer*. Boston, MA: Kluwer Academic Publishers, 1997.
- [5] B. Wong, “Points of view: Color blindness,” *Nature Methods*, vol. 8, no. 6, p. 441, 2011, doi: 10.1038/nmeth.1618.
- [6] Coblis Colour Blindness Simulator. [Online]. Available: <https://www.color-blindness.com/coblis-color-blindness-simulator/> [Accessed 3 Jun 2026].
- [7] [Author], “HEAT 2: Console Implementation and Feature Improvements,” Unpublished project report, University of Kent, School of Computing, [Year].
- [8] Equality Act 2010, c. 15, UK Parliament. [Online]. Available: <https://www.legislation.gov.uk/ukpga/2010/15/contents> [Accessed Jun 2026].
- [9] The Public Sector Bodies (Websites and Mobile Applications) (No. 2) Accessibility Regulations 2018, SI 2018/952, UK Government. [Online]. Available: <https://www.legislation.gov.uk/uksi/2018/952/contents/made> [Accessed Jun 2026].
- [10] World Wide Web Consortium, “Web Content Accessibility Guidelines (WCAG) 2.1,” W3C Recommendation, June 2018. [Online]. Available: <https://www.w3.org/TR/WCAG21/> [Accessed Jun 2026].
- [11] V. Potluri, P. Vaithilingam, S. Iyengar, Y. Ransalu, S. Oney, and J. P. Bigham, “CodeTalk: Improving Programming Environment Accessibility for Visually Impaired Developers,” in *Proc. CHI 2018*, ACM, 2018, doi: 10.1145/3173574.3173609.
- [12] C. M. Baker, L. R. Milne, and R. E. Ladner, “StructJumper: A Tool to Help Blind Programmers Navigate and Understand the Structure of Code,” in *Proc. CHI 2015*, ACM, 2015, doi: 10.1145/2702123.2702589.
- [13] A. Stefik and S. Siebert, “An Empirical Investigation into Programming Language Syntax,” *ACM Transactions on Computing Education*, vol. 13, no. 4, art. 19, 2013, doi: 10.1145/2534973.
- [14] D. R. Flatla and C. Gutwin, “So, About That Colour Vision Paper: Colour Vision Deficiency Simulation Tools and Developer Understanding,” in *Proc. ASSETS 2012*, ACM, 2012. [Verify full title before publication.]

APPENDIX

APPENDIX: UML DIAGRAMS

The PlantUML source files for Figures 1–3 are at `heat/documentation/diagram_class.puml`, `heat/documentation/diagram_sequence.puml`, and `heat/documentation/diagram_activity.puml`. They can be rendered with PlantUML or the PlantUML VS Code extension.

- **Figure 1** — Class diagram of the accessibility extension (see `diagram_class.puml`).
- **Figure 2** — Sequence diagram: TTS path from GHCI output to speech (see `diagram_sequence.puml`).
- **Figure 3** — Activity diagram: `KeyboardOpenFileDialog` interaction flow (see `diagram_activity.puml`).